

# Full-Stack Intern Training — Day 7

MODULE  
React Hooks & Side Effects

AUTHOR  
Penumuru Madhu Sudhan Reddy

TRACK  
90-Day Full-Stack Web Dev

**Core Objective:** Master side effect management with `useEffect`, understand the synchronization mental model, dependency arrays, cleanup functions, mutable references with `useRef`, referential stability with `useMemo` / `useCallback`, and build generic custom hooks ( `useFetch`, `useDebounce`, `useLocalStorage` ).

## 1. MENTAL MODEL OF USEEFFECT: SYNCHRONIZATION VS LIFECYCLE

In modern React, `useEffect` does NOT represent lifecycle hooks (mount/update/unmount). It represents **synchronization** with outside systems (network APIs, browser DOM, document title, timers).

```
// Synchronizing browser title to active floor
function FloorTracker(){
  const [floor,setFloor]=useState(1);

  useEffect(()=>{
    document.title=`BuildTrack - Floor ${floor}`;
    console.log("Updated title to Floor", floor);
  },[floor]); // Runs whenever 'floor' changes

  return <button onClick={()=>setFloor(f=>f+1)}>Next Floor ({floor})</button>;
}
```

CONSOLE & BROWSER OUTPUT (O/P)

On initial render: Title becomes "BuildTrack - Floor 1" | Console: "Updated title to Floor 1"  
After button click: Title becomes "BuildTrack - Floor 2" | Console: "Updated title to Floor 2"

## 2. DEPENDENCY ARRAY VARIATIONS & THE INFINITE LOOP TRAP

| Variation         | Syntax                                           | When Does the Effect Run?                                       |
|-------------------|--------------------------------------------------|-----------------------------------------------------------------|
| No array          | <code>useEffect(() =&gt; { ... })</code>         | After <b>every single render</b>                                |
| Empty array       | <code>useEffect(() =&gt; { ... }, [])</code>     | <b>Once</b> after initial component mount                       |
| With dependencies | <code>useEffect(() =&gt; { ... }, [a, b])</code> | After mount + whenever <code>a</code> or <code>b</code> changes |

**⚠ Infinite Loop Danger:**  
If you call `setCount(count + 1)` inside an effect with *no dependency array*, the state update triggers a re-render, which triggers the effect again forever!

## 3. CLEANUP FUNCTION: TIMERS & NETWORK INFLIGHT REQUESTS

Return a cleanup function to clear intervals, remove listeners, or abort fetch requests when component unmounts.

```
useEffect(()=>{
  const timer=setInterval(()=>console.log("Tick"),1000);
  return ()=>clearInterval(timer); // Prevents memory leaks!
},[]);
```

#### 4. USEREF: MUTABLE STORAGE WITHOUT RE-RENDERS & DOM ACCESS

| Feature              | useState                             | useRef                                            |
|----------------------|--------------------------------------|---------------------------------------------------|
| Purpose              | Data that directly influences the UI | Mutable data that doesn't trigger re-renders      |
| Re-render on change? | <b>YES</b>                           | <b>NO</b>                                         |
| Access               | <code>value</code>                   | <code>ref.current</code>                          |
| DOM Access           | No                                   | Yes (e.g. <code>inputRef.current.focus()</code> ) |

```
function AutoFocusInput(){
  const inputRef=useRef<HTMLInputElement|null>(null);
  useEffect(()=>{
    inputRef.current?.focus(); // Focuses input on mount!
  },[]);
  return <input ref={inputRef} placeholder="Site task..." />;
}
```

#### 5. USEMEMO & USECALLBACK: PERFORMANCE OPTIMIZATION

- `useMemo` : Caches the *result* of an expensive calculation.
- `useCallback` : Caches the *function reference* itself to prevent child re-renders.

```
const total=useMemo(()=>{
  return items.reduce((sum,item)=>sum+item.cost,0);
},[items]); // Only recalculates when items array changes!
```

#### 6. CUSTOM HOOKS: SIMPLE BEGINNER IMPLEMENTATIONS

##### Exercise 1: useFetch Hook

```
export function useFetch<T>(url: string){
  const [data,setData]=useState<T|null>(null);
  const [loading,setLoading]=useState(true);
  const [error,setError]=useState<string|null>(null);

  useEffect(()=>{
    if(!url){ setLoading(false); return; }
    setLoading(true);
    fetch(url)
      .then(res=>res.json())
      .then(d=>{ setData(d); setLoading(false); })
      .catch(e=>{ setError(e.message); setLoading(false); });
  },[url]);

  return { data,loading,error };
}
```

##### USEFETCH OUTPUT (O/P)

Initial: { data: null, loading: true, error: null }

After resolution: { data: { name: "Apex Tower A" }, loading: false, error: null }

Exercise 3: useDebounce Hook

```
export function useDebounce<T>(value: T, delay: number): T {
  const [debounced, setDebounced] = useState<T>(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}
```

**USEDEBOUNCE OUTPUT (O/P)**

User types "C" → "Co" → "Con" within 200ms.  
Timer resets on every keystroke. Effect fires ONCE after 300ms pause with "Con"!

Exercise 4: useLocalStorage Hook

```
export function useLocalStorage<T>(key: string, initial: T): [T, (val: T) => void] {
  const [val, setVal] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initial;
    } catch {
      return initial;
    }
  });

  const setValue = (newVal: T) => {
    setVal(newVal);
    try { window.localStorage.setItem(key, JSON.stringify(newVal)); } catch {}
  };

  return [val, setValue];
}
```

**USELOCALSTORAGE OUTPUT (O/P)**

User clicks [List View] → writes to localStorage.  
On page refresh → automatically loads 'list' from browser storage!

7. DAY 7 VERIFICATION CHECKLIST

| Check                  | Command                           | Expected Result                        |
|------------------------|-----------------------------------|----------------------------------------|
| TypeScript Compilation | <code>npm run typecheck</code>    | 0 errors, all types pass               |
| Unit Test Suite        | <code>npm run day7:test</code>    | 100 activities, debounce, storage pass |
| Interactive Demo       | Open <code>day7/index.html</code> | Live reactive UI in browser            |